

CerbaSeal-Core

v0.1.0 — System & Repository Breakdown

MAXIMUM DETAIL TECHNICAL REFERENCE

Generated: Fri, 05 Jun 2026 20:29:47 GMT
Jesse Lamont · jesse@lamontlabs.io

391 / 391

Test Suite

16 files · zero regressions

15 / 15

Repo Audit

all checks passing

INV-01 !' 12

Invariants

100% test coverage

0 violations

Import Bounds

79 files scanned

CONTENTS

- 1 System Purpose & Architecture Overview
- 2 Domain Type System — All Interfaces & Enums
- 3 Invariant Registry (INV-01 !' INV-12) — Full Detail
- 4 Reason Code Registry — All 18 Codes
- 5 Execution Gate Service — Full Logic Flow
- 6 Authority Class System & Runtime Config
- 7 Audit & Evidence Pipeline
- 8 Supporting Services (Replay, Diagnostics, Export)
- 9 Test Suite — 16 Files, 391 Tests
- 10 Repo Audit — 15 Checks Explained
- 11 CLI Scripts & Commands
- 12 Integration Starter Kits (×4)
- 13 Adoption Layer — 5 Priorities
- 14 Demo App Routes
- 15 Full Repository File Tree

What CerbaSeal-Core Is

CerbaSeal-Core is a deterministic execution governance infrastructure library for AI-integrated financial and enterprise workflows. It acts as a mandatory gate that sits between any AI proposal and any consequential action. No action — payment, hold, escalation, account modification — may be released without passing through the gate. The gate is not advisory: it issues ALLOW, HOLD, or REJECT with a cryptographically chained audit trail that cannot be altered after the fact.

Core Design Principles

Fail-closed by default: Any unexpected error, malformed request, or missing field produces REJECT. There is no silent allow path.

AI is proposal-only: An AI actor can submit a proposal but cannot bear authority. Attempting self-authorization produces an unconditional REJECT.

Human approval as a release precondition: For workflows requiring human approval, no ALLOW is issued until a valid, signed, correctly-bound ApprovalArtifact is present.

Immutable decision envelopes: Every GateResult is stamped with an immutable: true flag and registered in a module-private WeakSet. EvidenceBundleService rejects any result not in that set.

Cryptographic hash chain: Every audit log entry includes a SHA-256 hash of its own payload and the hash of the previous entry, forming a tamper-evident chain verifiable at any time.

Deterministic replay: Any past decision can be fed back through the gate (ReplayService) to verify that the same inputs produce the same output — proving no logic drift.

Zero external dependencies (core): The enforcement core has no runtime npm dependencies beyond Node.js built-ins. pdfkit and tsx are devDependencies only.

Architecture Layers

Layer	Location	Responsibility
Domain Layer	src/domain/	Types, constants, errors, formatters, builders. No business logic — pure structural definitions.
Enforcement Layer	src/services/execution/	ExecutionGateService — the single mandatory gate. assertIsGateIssued() — WeakSet bypass prevention.
Audit Layer	src/services/audit/	AppendOnlyLogService (in-memory). FileBackedAppendOnlyLogService (JSONL persistence). audit-hash-utils.ts — SHA-256 chain.
Evidence Layer	src/services/evidence/	EvidenceBundleService — assembles full decision context from gate result + audit entries.
Export Layer	src/services/export/	ExportManifestService — point-in-time snapshot of evidence hashes for external transfer.
Replay Layer	src/services/replay/	ReplayService — re-runs past decisions through the gate to verify determinism.
Diagnostics Layer	src/services/diagnostics/	DiagnosticReportService — operator-facing state analysis with recommended actions.
Support Layer	src/services/support/	OperatorActionService, SystemHealthService, SystemIntegrityService.
Config Layer	src/config/	cerbaseal-config.ts — loadCerbaSealConfig(), buildAllowedAuthorityClasses(). Reads cerbaseal.config.json.
Examples Layer	examples/	Working demos: browser-demo (HTTP server), consumer, agent-gate, auditor-view. sunnort-readiness. 4 starter kits.
Scripts Layer	scripts/	repo-audit, export-proof, verify-proof, check-imports, check-invariants, generate-pilot-config, generate-evidence-report.

Technology Stack

Component	Choice	Notes
Runtime	Node.js 18+	ESM module system ("type": "module" in package.json)
Language	TypeScript 5.6	Strict mode, ES2022 target, NodeNext module resolution
Test runner	Vitest 2.1	391 tests, 16 files, avg 7.7s full run
Script runner	tsx 4.21	Direct TypeScript execution without compile step
PDF generation	pdfkit 0.18	devDependency — used only for report generation
Package manager	pnpm	Workspace monorepo with cerbaseal-demo artifact
Demo UI	Vite + React	Browser-based review portal at /review, /pilot, /security

Primitive Union Types (src/domain/types/core.ts)

Type	Values	Notes
AuthorityClass	system ai analyst reviewer manager compliance_officer	Who is making the request. Extensible via config.
HumanAuthorityClass	analyst reviewer manager compliance_officer	Subset used for approval artifacts — excludes svstem/ai.
WorkflowClass	fraud_triage transaction_escalation account_hold_recommendation	Which workflow envelope governs this request.
ActionClass	allow hold reject escalate account_hold	Concrete action the gate may permit.
UnknownableActionClass	ActionClass (string & {})	Input type — gate validates and narrows to ActionClass or REJECTs.
ProposalSourceKind	ai deterministic_rule	Origin of the proposal. Does not grant authority.
DecisionState	ALLOW HOLD REJECT	Final gate output. Produced once, immutable.

Core Interfaces

GovernedRequest · core.ts

Primary input to ExecutionGateService.evaluate(). Every field is validated before a decision is produced.

• r
e
q
u
es
tl
d:
str
in
g
—
n
o
n-
e
m
pt
y
u
ni
q
u
e i
d
e
nti
fie
r f
or
t
hi
s r
e
q
u
es
t
•

w
or
kfl
o
w
Cl
as
s:
W
or
kfl
o
w
Cl
as
s
—
d
et
er
mi
n
es
w
hi
ch
in
va
ri
a
nt
s
a
p
pl
y
• j
ur
is
di
cti
o
n:
str
in
g
—
n
o
n-
e
m
pt
y j
ur
is
di
cti
o
n
al
ta

g

•

ac

to

rl

d:

str

in

g

—

o

p

a

q

u

e

ac

to

r i

d

e

nti

fie

r

•

ac

to

rA

ut

h

or

ity

Cl

as

s:

A

ut

h

or

ity

Cl

as

s

—

v

ali

d

at

e

d

at

r

u

nti

m

e

a

g

ai

ns

t

all

o
w
e
d
se
t
•
pr
o
p
os
e
d
A
cti
o
n
Cl
as
s:
U
nk
n
o
w
n
a
bl
e
A
cti
o
n
Cl
as
s
—
m
us
t
m
at
ch
p
ro
p
os
al.
re
q
u
es
te
d
A
cti
o
n
Cl
as
s
•

propositional — the Al/rule propositional belief evaluation • sensitive : boolean — if true, state control stori

g
g
er
R
E
J
E
C
T

•
pr
o
hi
bit
e
d
U
se
: b
o
ol
e
a
n

—
if
tr
u
e,
u
nc
o
n
dit
io
n
al
R
E
J
E
C
T

•
p
oli
cy
P
ac
k
R
ef
:
P
oli
cy
P
ac
k
R
ef
|

nu
ll
—
m
us
t
b
e
n
o
n-
nu
ll t
o
pr
oc
e
e
d
•
pr
ov
e
n
a
nc
e
R
ef
:
Pr
ov
e
n
a
nc
e
R
ef
|
nu
ll
—
m
us
t
b
e
n
o
n-
nu
ll
wi
th
al
l fi
el
ds
•
a
p

pr
ov
al
R
e
q
uir
e
d:
b
o
ol
e
a
n
—
c
all
er
-s
u
p
pli
e
d
fla
g
(s
o
m
e
w
or
kfl
o
w
s
ov
er
ri
d
e)
•
a
p
pr
ov
al
Ar
tif
ac
t:
A
p
pr
ov
al
Ar
tif
ac
t |
nu

II
—
m
us
t
b
e
pr
es
e
nt
if
a
p
pr
ov
al
re
q
uir
e
d
• I
o
g
gi
n
g
R
e
a
dy
: b
o
ol
e
a
n
—
m
us
t
b
e
tr
u
e
to
p
ro
ce
e
d
•
co
nt
rol
St
at
us
:
C

o
nt
rol
St
at
us

—
st
al
e
ch
ec
k f
or
s
e
ns
iti
ve
fl
o
w
s
• t
ru
st
St
at
e:
Tr
us
tS
ta
te

—
m
us
t
b
e
tr
us
te
d
to
p
ro
ce
e
d
•
cr
e
at
e
d
At
: s
tri
n

g
—
I
S
O
ti
m
es
ta
m
p
va
lid
at
e
d
a
g
ai
ns
t
a
p
pr
ov
al
ti
m
es
ta
m
p

DecisionEnvelope · **core.ts**

Immutable output produced by the gate. Registered in WeakSet. EvidenceBundleService rejects non-registered envelopes.

•
e
nv
el
o
p
el
d:
str
in
g
—
d
et
er
mi
ni
sti
c:
'e
nv
'
—
+
re
q
u
es

tl
d
• r
e
q
u
es
tl
d:
str
in
g
—
b
ac
k-
re
fe
re
nc
e
to
t
h
e
re
q
u
es
t
•
w
or
kfl
o
w
Cl
as
s:
W
or
kfl
o
w
Cl
as
s
—
e
ch
o
e
d
fr
o
m
re
q
u
es
t
• f

in
al
St
at
e:
D
ec
isi
o
n
St
at
e
—
A
L
L
O
W
|
H
O
L
D
|
R
E
J
E
C
T
.
p
er
mi
tt
e
d
A
cti
o
n
Cl
as
s:
A
cti
o
n
Cl
as
s
|n
ull

—
s
et
o
n
A
L

L
O
W
;n
ull
o
n
H
O
L
D/
R
E
J
E
C
T
•
h
u
m
a
n
A
p
pr
ov
al
R
e
q
uir
e
d:
b
o
ol
e
a
n
—
e
ch
o
e
d
fr
o
m
re
q
u
es
t
•
h
u
m
a
n
A
p

pr
ov
al
Pr
es
e
nt
: b
o
ol
e
a
n
—
tr
u
e
if
a
p
pr
ov
al
Ar
tif
ac
t
w
as
p
re
se
nt
•
pr
o
p
os
al
S
o
ur
ce
Ki
n
d:
Pr
o
p
os
al
S
o
ur
ce
Ki
n
d
—
e
ch
o

e
d
fr
o
m
pr
o
p
os
al
• i
m
m
ut
a
bl
e:
tr
u
e
—
lit
er
al
ty
p
e,
al
w
ay
s t
ru
e
•
ev
id
e
nc
e
B
u
n
dl
el
d:
str
in
g
—
d
et
er
mi
ni
sti
c:
'e
vi
d
e
nc
e

—
+
re
q
u
es
tl
d
• t
ra
ce
:
D
ec
isi
o
n
Tr
ac
e
—
c
h
ec
ke
dl
nv
ar
ia
nt
s[]
, r
e
as
o
n
C
o
d
es
[],
e
va
lu
at
e
d
At
• i
ss
u
e
d
At
: s
tri
n
g
—
I
S
O

ti
m
es
ta
m
p
of
g
at
e
ev
al
u
ati
o
n

ApprovalArtifact · core.ts

Human approval record. Must be bound to a specific requestId. Timestamp must postdate request.createdAt.

- a
p
pr
ov
all
d:
str
in
g
—
u
ni
q
u
e
a
p
pr
ov
al
id
e
nti
fie
r
- a
p
pr
ov
er
Id
: s
tri
n
g
—
h
u
m
a
n

re
vi
e
w
er
id
e
nti
ty
• f
or
R
e
q
u
es
tl
d:
str
in
g
—
m
us
t
e
q
u
al
G
ov
er
n
e
d
R
e
q
u
es
t.r
e
q
u
es
tl
d
•
a
p
pr
ov
er
A
ut
h
or
ity
Cl
as
s:
H

u
m
a
n
A
ut
h
or
ity
Cl
as
s
—
m
us
t
b
e
a
n
al
ys
t|r
ev
ie
w
er
|m
a
n
a
g
er
|c
o
m
pli
a
nc
e
—
of
fic
er
•
pr
ivi
le
g
e
d
A
ut
h
S
ati
sfi
e
d:
b
o
ol

e
a
n
—
m
us
t
b
e
tr
u
e
• i
m
m
ut
a
bl
e
Si
g
n
at
ur
e:
str
in
g
—
n
o
n-
e
m
pt
y
si
g
n
at
ur
e
str
in
g
•
a
p
pr
ov
e
d
At
: s
tri
n
g
—
I
S
O
ti

m
es
ta
m
p;
m
us
t
b
e
>
=
re
q
u
es
t.c
re
at
e
d
At

EvidenceBundle · audit.ts

Full decision snapshot assembled by EvidenceBundleService. Contains the original request, envelope, release/blocked records, and full hash-linked event chain.

•
ev
id
e
nc
e
B
u
n
dl
el
d:
str
in
g
—
m
at
ch
es
d
ec
isi
o
n
E
nv
el
o
p
e.
ev
id
e
nc
e

B
u
n
dl
el
d
• r
e
q
u
es
t:
G
ov
er
n
e
d
R
e
q
u
es
t
—
d
e
e
p-
cl
o
n
e
d
or
igi
n
al
re
q
u
es
t
•
d
ec
isi
o
n
E
nv
el
o
p
e:
D
ec
isi
o
n
E
nv

el
o
p
e
—
d
e
e
p-
cl
o
n
e
d
e
nv
el
o
p
e
• r
el
e
as
e
A
ut
h
or
iz
ati
o
n:
R
el
e
as
e
A
ut
h
or
iz
ati
o
n
| n
ull
—
p
re
se
nt
o
n
A
L
L
O
W
•

bl
oc
ke
d
A
cti
o
n
R
ec
or
d:
Bl
oc
ke
d
A
cti
o
n
R
ec
or
d
| n
ull

—
p
re
se
nt
o
n
H
O
L
D/
R
E
J
E
C
T
•
ev
e
nt
C
h
ai
n:
A
u
dit
L
o
g
E
nt
ry
[]

—
al
ll
o
g
e
nt
ri
es
fo
rt
hi
sr
e
q
u
es
tl
d
•
cr
e
at
e
d
At
:s
tri
n
g
—
I
S
O
ti
m
es
ta
m
p
of
b
u
n
dl
e
cr
e
ati
o
n

AuditLogEntry · audit.ts

Single entry in the tamper-evident hash chain. Every entry hashes its own payload and links to the previous entry hash.

•
ev
e
nt
ld
:s
tri
n

g
—
u
ni
q
u
e
e
nt
ry
id
e
nti
fie
r
• r
e
q
u
es
tl
d:
str
in
g
—
r
e
q
u
es
t t
hi
s
ev
e
nt
b
el
o
n
gs

to
•
ev
e
nt
Ty
p
e:
A
u
dit
E
ve
nt
Ty
p
e
—
R

E
Q
U
E
S
T
-
E
V
A
L
U
A
T
E
D
|
R
E
L
E
A
S
E
-
A
U
T
H
O
R
I
Z
E
D
|
A
C
T
I
O
N
-
B
L
O
C
K
E
D
|
E
V
I
D
E
N
C
E
-
B
U
N
D
L

E
-
C
R
E
A
T
E
D
|
E
X
P
O
R
T
-
M
A
N
I
F
E
S
T
-
C
R
E
A
T
E
D
• t
im
es
ta
m
p:
str
in
g
—
I
S
O
ti
m
es
ta
m
p
•
p
ay
lo
a
d
H
as
h:
str
in

g
—
S
H
A-
2
5
6
of
t
h
e
ev
e
nt
p
ay
lo
a
d
J
S
O
N
•
pr
ev
io
us
H
as
h:
str
in
g
| n
ull

—
n
ull
fo
r
g
e
n
es
is
e
nt
ry
; h
as
h
of
p
re
vi
o
us
e
nt

ry
o
th
er
wi
se
•
e
nt
ry
H
as
h:
str
in
g
—
S
H
A-
2
5
6
of
(
p
ay
lo
a
d
H
as
h
+
pr
ev
io
us
H
as
h
+
ti
m
es
ta
m
p)

SECTION 3 — INVARIANT REGISTRY (INV-01 !' INV-12) — FULL DETAIL

Invariants are the non-negotiable rules of the system. Every check is applied on every `evaluate()` call. A failed invariant throws a `CerbaSealError` (or produces a controlled `REJECT` result) — there is no path to `ALLOW` that skips any invariant. All 12 invariants have explicit test coverage confirmed by `scripts/check-invariant-coverage.ts` (run as part of `pnpm audit:repo`).

INV-01 NO_POLICY_PACK_NO_EXECUTION !' REJECT

Every governed request must carry a reference to the policy pack under which it was evaluated. A request with no policy pack cannot proceed — there is no basis for determining what rules govern the action.

Trigger: `policyPackRef === null`

Reason Code: `NO_POLICY_PACK`

Test reference: `execution-gate-service.test.ts` — 'rejects when no policy pack'

INV-02 NO_PROVENANCE_NO_ACTION !' REJECT

Every request must carry full provenance: model version, rule set version, and source hash. Empty provenance is treated as missing. Without provenance, the decision cannot be audited or replayed deterministically.

Trigger: `provenanceRef === null` or any field empty

Reason Code: `NO_PROVENANCE`

Test reference: `execution-gate-service.test.ts` — 'rejects when no provenance'

INV-03 NO_REQUIRED_APPROVAL_NO_RELEASE !' HOLD (missing), REJECT (invalid/forged)

When approval is required (caller-set OR workflow-class-forced), no `ALLOW` is issued until a valid `ApprovalArtifact` is present. The artifact must: (a) be bound to this specific `requestId`, (b) have a timestamp `"e request.createdAt`, (c) carry a valid `HumanAuthorityClass`, (d) have `privilegedAuthSatisfied: true`, (e) have a non-empty `immutableSignature`.

Trigger: `effectiveApprovalRequired && approvalArtifact === null`

Reason Code: `REQUIRED_APPROVAL_MISSING` / `INVALID_APPROVAL_AUTHORITY`

Test reference: `execution-gate-service.test.ts` — approval binding, timestamp, forgery tests

INV-04 NO_LOGGING_NO_EXECUTION !' REJECT

Before any action is released, the logging subsystem must be confirmed ready. This prevents situations where actions are taken but not recorded — a governance failure equivalent to acting without a paper trail.

Trigger: `loggingReady === false`

Reason Code: `LOGGING_NOT_READY`

Test reference: `execution-gate-service.test.ts` — 'rejects when logging not ready'

SECTION 3 — INVARIANT REGISTRY (CONTINUED)

INV-05 AI_NON_AUTHORITATIVE != REJECT

AI actors are proposal-only. Two separate checks: (1) if the proposal itself claims authority-bearing status, REJECT; (2) if the actor is 'ai' AND the proposal source is 'ai', REJECT — regardless of the approvalRequired flag. Fix 4 closed a prior bypass where setting approvalRequired: false allowed an AI actor through.

Trigger: proposal.authorityBearing === true OR (actorAuthorityClass === 'ai' AND proposalSourceKind === 'ai')

Reason Code: AI_CANNOT_AUTHORIZE

Test reference: security/misuse-scenarios.test.ts, adversarial-integrity.test.ts

INV-06 NO_BYPASS_OF_EXECUTION_GATE != REJECT (CerberaSealError thrown)

A module-private WeakSet (_gateIssuedResults) registers every GateResult produced by ExecutionGateService.evaluate(). EvidenceBundleService calls assertIsGateIssued() before accepting any result. A self-constructed GateResult cannot pass this check — the WeakSet is inaccessible outside the module.

Trigger: EvidenceBundleService receives a GateResult not in _gateIssuedResults WeakSet

Reason Code: MALFORMED_REQUEST

Test reference: security/non-forgery.test.ts — 'rejects self-constructed gate result'

INV-07 IMMUTABLE_DECISION_ENVELOPE != Structural enforcement

DecisionEnvelope carries immutable: true as a literal TypeScript type. EvidenceBundleService deep-clones all objects before returning them. The envelope is never mutated after construction. This invariant is checked in the trace even on ALLOW paths.

Trigger: Always — structural guarantee

Reason Code: N/A — compile-time + runtime deep-clone

Test reference: execution-gate-service.test.ts — immutable flag assertions

INV-08 STALE_CONTROLS_BLOCK_SENSITIVE_RELEASE != REJECT

For sensitive requests, the control status must be both current (not stale) and valid. This ensures that time-sensitive compliance controls (e.g. sanction list checks, credit limit verifications) cannot be bypassed by using an old control state.

Trigger: request.sensitive === true AND (controlStatus.stale OR !controlStatus.criticalControlsValid)

Reason Code: CONTROL_STALE_OR_INVALID

Test reference: execution-gate-service.test.ts — stale controls tests

SECTION 3 — INVARIANT REGISTRY (CONTINUED)

INV-09 TRUST_STATE_REQUIRED != REJECT

Every request must carry a valid trust state. If the system's trust evaluation has determined the request context is not trusted (e.g. flagged actor, revoked credentials), the gate rejects unconditionally — before approval state is even checked.

Trigger: `trustState.trusted === false`

Reason Code: `TRUST_STATE_INVALID`

Test reference: `execution-gate-service.test.ts` — 'rejects invalid trust state'

INV-10 PROHIBITED_USE_MUST_BLOCK != REJECT

If the request has been flagged as prohibited use (e.g. sanctioned jurisdiction, restricted action type, policy exclusion), the gate rejects unconditionally. This check runs after trust state but before approval state to ensure compliance blocks are absolute.

Trigger: `request.prohibitedUse === true`

Reason Code: `PROHIBITED_USE`

Test reference: `execution-gate-service.test.ts` — 'rejects prohibited use'

INV-11 REQUEST_SCHEMA_AND_ACTION_CLASS_VALID != REJECT

Shape validation runs first, before any semantic checks. Catches: empty required string fields, unknown action classes (not in `ALLOWED_ACTION_CLASSES` set), unknown authority classes (not in `allowedAuthorityClasses` set), mismatched `proposedActionClass` vs `proposal.requestedActionClass`. TypeScript enforces types at compile time; this closes the runtime gap for untyped callers.

Trigger: `requestId` empty | `jurisdiction` empty | `createdAt` empty | `reasonCodes` empty | unknown action class | unknown authority class

Reason Code: `MALFORMED_REQUEST` / `UNKNOWN_ACTION_CLASS`

Test reference: `execution-gate-service.test.ts` — schema validation suite

INV-12 PROPOSAL_AND_REQUEST_ACTION_MUST_MATCH != REJECT

The action class at the request level and the action class inside the proposal must be identical. This prevents a request from claiming one action type externally while the proposal contains a different action type — a potential vector for action-class confusion attacks.

Trigger: `proposedActionClass !== proposal.requestedActionClass`

Reason Code: `INVALID_PROPOSAL`

Test reference: `execution-gate-service.test.ts` — 'rejects mismatched action classes'

SECTION 4 — REASON CODE REGISTRY — ALL 18 CODES

Reason codes are emitted into `DecisionEnvelope.trace.reasonCodes`. Every ALLOW, HOLD, and REJECT decision carries at least one reason code. HOLD and REJECT decisions always include a terminal code (`DECISION_HELD` or `DECISION_REJECTED`) in addition to the trigger code.

Code	Final State	Invariant	Meaning
<code>DECISION_ALLOWED</code>	ALLOW	—	All checks passed. Action is released.
<code>DECISION_HELD</code>	HOLD	—	Terminal suffix on all HOLD outcomes.
<code>DECISION_REJECTED</code>	REJECT	—	Terminal suffix on all REJECT outcomes.
<code>NO_POLICY_PACK</code>	REJECT	INV-01	<code>policyPackRef</code> is null.
<code>NO_PROVENANCE</code>	REJECT	INV-02	<code>provenanceRef</code> is null or any field is empty.
<code>REQUIRED_APPROVAL_MISSING</code>	HOLD	INV-03	Approval required but <code>approvalArtifact</code> is null.
<code>INVALID_APPROVAL_AUTHORITY</code>	REJECT	INV-03	Artifact bound to wrong request, or invalid authority class, or timestamp predates request.
<code>PRIVILEGED_AUTH_NOT_SATISFIED</code>	REJECT	INV-03	<code>privilegedAuthSatisfied</code> is false.
<code>APPROVAL_SIGNATURE_MISSING</code>	REJECT	INV-03	<code>immutableSignature</code> is empty.
<code>LOGGING_NOT_READY</code>	REJECT	INV-04	<code>loggingReady</code> is false.
<code>AI_CANNOT_AUTHORIZE</code>	REJECT	INV-05	AI actor or authority-bearing proposal attempted authorization.
<code>PROHIBITED_USE</code>	REJECT	INV-10	<code>prohibitedUse</code> flag is true.
<code>CONTROL_STALE_OR_INVALID</code>	REJECT	INV-08	Sensitive request with stale or invalid control status.
<code>TRUST_STATE_INVALID</code>	REJECT	INV-09	<code>trustState.trusted</code> is false.
<code>UNKNOWN_ACTION_CLASS</code>	REJECT	INV-11	<code>proposedActionClass</code> or <code>proposal.requestedActionClass</code> is not a known <code>ActionClass</code> .
<code>MALFORMED_REQUEST</code>	REJECT	INV-11/06	Empty required field, or self-constructed <code>GateResult</code> , or non-ISO timestamp.
<code>INVALID_PROPOSAL</code>	REJECT	INV-12	<code>proposedActionClass</code> != <code>proposal.requestedActionClass</code> .
<code>AI_CANNOT_AUTHORIZE (auth-bearing)</code>	REJECT	INV-05	<code>proposal.authorityBearing</code> is true (regardless of actor class).

evaluate(request: GovernedRequest): GateResult

This is the single mandatory entry point for all governed actions. The method runs a strict sequence of invariant assertions. The first failure short-circuits evaluation and produces a controlled HOLD or REJECT result. On success, ALLOW is issued and registered in the module-private WeakSet.

Evaluation Sequence (in order)

Step	Function	Invariant	On fail	What is checked
1	assertRequestShape()	INV-11	REJECT	Validates requestId, jurisdiction, createdAt non-empty; proposal.reasonCodes non-empty
2	assertActorAuthorityClass()	INV-11	REJECT	Validates actorAuthorityClass against allowedAuthorityClasses set (runtime check)
3	assertKnownActionClass(proposedActionClass)	INV-11	REJECT	Must be one of: allow, hold, reject, escalate, account_hold
4	assertKnownActionClass(proposal.requestedActionClass)	INV-11	REJECT	Same check on the proposal-side action class
5	proposal action === request action	INV-12	REJECT	proposedActionClass must identically match proposal.requestedActionClass
6	assertPolicyPack()	INV-01	REJECT	policyPackRef must be non-null
7	assertProvenance()	INV-02	REJECT	provenanceRef non-null; modelVersion, ruleSetVersion, sourceHash all non-empty
8	assertLoggingReady()	INV-04	REJECT	loggingReady must be true
9	assertProposalBoundary()	INV-05	REJECT	Not authority-bearing; AI actor+AI proposal always rejects
10	assertProhibitedUse()	INV-10	REJECT	prohibitedUse must be false
11	assertControlStatus()	INV-08	REJECT	Sensitive request: controls must be current and valid
12	assertTrustState()	INV-09	REJECT	trustState.trusted must be true
13	assertApprovalState()	INV-03	HOLD/REJECT	Effective approval required? Artifact present, bound, timestamped, signed, valid authority?
14	buildDecisionEnvelope(ALLOW)	INV-07	ALLOW	Construct immutable envelope; push to WeakSet; build ReleaseAuthorization; return GateResult

Error Handling — Fail-Closed Guarantee

The try/catch at the top of evaluate() guarantees two things:

•
C
er
b
a
S
e
al
Er
ro
r (
ex
p
ec
te
d
fail
ures
)
pr
o
d
uc
es

a
cl
e
a
n
D
ec
isi
o
n
E
nv
el
o
p
e
wi
th
t
h
e
er
ro
r's
fi
n
al
St
at
e
(H
O
L
D
or
R
E
J
E
C
T)
,t
h
e
tri
g
g
er
r
e
as
o
n
co
d
e,
a
n
d
th
e

terminal code.
• Non-Cerbera Seal Error (unexpected exception): converted to a REJECT DECISION envelope with

M
A
L
F
O
R
M
E
D
-
R
E
Q
U
E
S
T
+
D
E
C
I
S
I
O
N
-
R
E
J
E
C
T
E
D.

If
ev
e
n
thi
s f
all
b
ac
k f
ail
s (
re
q
u
es
t t
o
o
br
ok
e
n
to
r
e
a
d)

, t
h
e
or
igi
n
al
er
ro
r
is
re
-t
hr
o
w
n
—
st
ill
fai
l-c
lo
se
d
at
t
h
e
ca
lle
r
b
o
u
n
d
ar
y.

Deterministic ID Functions

All IDs are deterministic — replay produces identical identifiers for the same requestId

```
envelopeId      = 'env_'      + requestId  
evidenceBundleId = 'evidence_' + requestId  
releaseAuthId   = 'release_'  + requestId
```

WORKFLOWS_REQUIRING_APPROVAL (unconditional override)

Certain workflow classes require human approval regardless of the caller-supplied approvalRequired flag. The caller cannot opt out. Adding a workflow class to this set makes approval non-bypassable for that workflow.

```
const WORKFLOWS_REQUIRING_APPROVAL = new Set<WorkflowClass>([  
  'fraud_triage' // approval always required; caller flag is advisory only  
]);
```

assertIsGateIssued() — Bypass Prevention

A module-private WeakSet (`_gateIssuedResults`) is populated only inside `evaluate()`. `EvidenceBundleService` calls `assertIsGateIssued(gateResult)` before accepting any result. A caller who constructs their own `GateResult` cannot add it to the WeakSet — the reference is not exported.

```
const _gateIssuedResults = new WeakSet<object>(); // module-private, never exported  
  
export function assertIsGateIssued(result: GateResult): void {
```

```
if (!_gateIssuedResults.has(result)) {
```

```
throw new CerbaSealError({ ... finalState: 'REJECT' });
```


Core Authority Classes

Class	Kind	Can propose?	Can authorize?	Notes
system	System actor	Yes	Yes (non-AI path)	Internal system processes, deterministic rules
ai	AI actor	Yes (proposal only)	NEVER	INV-05: AI actor + AI proposal = unconditional REJECT
analyst	Human approver	Yes	Yes (as approver)	HumanAuthorityClass — valid approval authority
reviewer	Human approver	Yes	Yes (as approver)	HumanAuthorityClass — valid approval authority
manager	Human approver	Yes	Yes (as approver)	HumanAuthorityClass — valid approval authority
compliance_officer	Human approver	Yes	Yes (as approver)	HumanAuthorityClass — highest human authority class

Runtime Extensibility — cerbaseal.config.json

New client-specific authority classes (e.g. 'risk_officer', 'supervisor', 'credit_committee') can be added to cerbaseal.config.json without modifying any TypeScript source code. The config is loaded at ExecutionGateService construction time via loadCerbaSealConfig().

cerbaseal.config.json — current state

```
{
  "_comment": "Add client roles to extended[] – no TypeScript changes required.",
  "authorityClasses": {
    "core": ["system", "ai", "analyst", "reviewer", "manager", "compliance_officer"],
    "extended": []
  },
  "workflowClasses": {
    "core": ["fraud_triage", "transaction_escalation", "account_hold_recommendation"],
    "extended": []
  },
  "actionClasses": {
    "core": ["allow", "hold", "reject", "escalate", "account_hold"],
    "extended": []
  }
}
```

src/config/cerbaseal-config.ts — API Surface

```
interface GateConfig {
  additionalAuthorityClasses?: string[];
}

function loadCerbaSealConfig(): GateConfig
  // Reads cerbaseal.config.json from process.cwd()
  // Returns { additionalAuthorityClasses: config.authorityClasses.extended }
  // Falls back to {} if file not found (core classes only)

function buildAllowedAuthorityClasses(config?: GateConfig): ReadonlySet<string>
  // Returns Set of: CORE_AUTHORITY_CLASSES + (config?.additionalAuthorityClasses ?? [])
```

Usage Patterns

```
// Pattern 1: Default (core classes only – all existing tests unchanged)
```

```
const gate = new ExecutionGateService();
```



```
// Pattern 2: Inline extension
```

```
const gate = new ExecutionGateService({
```

additionalAuthorityClasses: ['risk_officer', 'supervisor']

});


```
// Pattern 3: From cerbaseal.config.json
```

```
import { loadCerbaSealConfig } from './src/config/cerbaseal-config.js';
```

```
const gate = new ExecutionGateService(loadCerberaSealConfig());
```


AppendOnlyLogService (in-memory)

Implements `IAuditLogService`. All entries are stored in a private array. The public API provides `append`, `list`, `listByRequestId`, and `verifyChain`. No entry can be deleted or modified via this API.

```
interface IAuditLogService {
  append(event: AuditEventPayload): AuditLogEntry;
  list(): AuditLogEntry[];
  listByRequestId(requestId: string): AuditLogEntry[];
  verifyChain(): boolean;
}
```

FileBackedAppendOnlyLogService (persistent)

Same interface. Writes every entry as a newline-delimited JSON (JSONL) record to a file path specified at construction. Entries are written synchronously on every `append` call (`appendFileSync`). On construction, reads existing entries from the file to restore state across restarts. Used by `fraud-workflow-starter` and any production deployment requiring durable audit logs.

```
const log = new FileBackedAppendOnlyLogService('./audit/fraud-triage.jsonl');
// File is created automatically if it does not exist.
// Existing entries are loaded and chain is verified on init.
```

Hash Chain Construction (audit-hash-utils.ts)

Each entry carries three hashes, all using SHA-256 (Node.js `crypto` module):

```
payloadHash = SHA256(JSON.stringify(eventPayload))
previousHash = entryHash of the previous entry (null for genesis)
entryHash = SHA256(payloadHash + '|' + (previousHash ?? 'genesis') + '|' + timestamp)

verifyChain() re-computes every entryHash and checks the previousHash chain.
Returns false if any entry has been tampered with or the chain is broken.
```

Events Written Per Request (EvidenceBundleService)

Event Type	When written	Key payload fields
REQUEST_EVALUATED	Always — on every <code>evaluate()</code> call	<code>requestId</code> , <code>finalState</code> , <code>envelopId</code>
RELEASE_AUTHORIZED	On <code>ALLOW</code> only	<code>requestId</code> , <code>releaseAuthorizationId</code> , <code>actionClass</code>
ACTION_BLOCKED	On <code>HOLD</code> or <code>REJECT</code>	<code>requestId</code> , <code>finalState</code> , <code>reasonCodes[]</code>
EVIDENCE_BUNDLE_CREATED	Always — after above events	<code>requestId</code> , <code>evidenceBundleId</code>

EvidenceBundleService.createBundle() — Full Flow

Steps executed in order:

-
- 1.
- C
- all
- a
- ss
- er
- tls
- G
- at
- el
- ss
- u

e
d(
g
at
e
R
es
ult
)
—
r
ej
ec
ts
se
lf-
co
ns
tr
uc
te
d
re
su
lts
(I
N
V-
0
6)
•
2.
A
p
p
e
n
d
R
E
Q
U
E
S
T
—
E
V
A
L
U
A
T
E
D
ev
e
nt
t
o
a
u

dit
lo
g
•
3.
If
rel
e
as
e
A
ut
h
or
iz
ati
o
n
pr
es
e
nt
: a
p
p
e
n
d
R
E
L
E
A
S
E
—
A
U
T
H
O
R
I
Z
E
D
ev
e
nt
•
4.
If
bl
oc
ke
d
A
cti
o
n
R
ec
or

d
pr
es
e
nt
: a
p
p
e
n
d
A
C
T
I
O
N
–
B
L
O
C
K
E
D
ev
e
nt
•
5.
A
p
p
e
n
d
E
VI
D
E
N
C
E
–
B
U
N
D
L
E
–
C
R
E
A
T
E
D
ev
e
nt
•
6.

Call
log
Service
List
By
Request
titles
d(re
ques
titles
d)
to
get
the
level
n
c
h
ai
n
for
his
re
ques
t
•
7.
Ass
emb
le
E
vid
enc

e
B
u
n
dl
e
wi
th
d
e
e
p-
cl
o
n
es
o
f
all
o
bj
ec
ts
(n
o
sh
ar
e
d
re
fe
re
nc
es
)
•
8.
R
et
ur
n
d
e
e
p-
cl
o
n
e
d
bu
n
dl
e
(d
o
u
bl
e-
cl
o
n

e
d
—
n
o
re
fe
re
nc
e
to
in
te
rn
al
st
at
e)

ExportManifestService

Creates a point-in-time snapshot of evidence hashes for external transfer (e.g. to a regulator, auditor, or archival system).

The manifest captures source event hashes but does not include the request payload — suitable for sharing with parties who need proof of decision without the underlying data.

```
interface ExportManifest {  
    manifestId: string;  
    evidenceBundleId: string;  
    requestId: string;  
    envelopeId: string;  
    exportedAt: string;  
    exportType: 'authority_package';  
    sourceEventHashes: string[];    // one per AuditLogEntry  
    sourceEvidenceImmutable: true;  // literal type  
}
```


ReplayService (src/services/replay/)

Takes a past EvidenceBundle and re-runs the original GovernedRequest through a fresh ExecutionGateService instance. Compares the replayed finalState against the original finalState. Used to verify: (a) the gate is deterministic, (b) logic has not drifted since the original decision, (c) the evidence bundle faithfully represents what happened.

```
interface ReplayResult {
  originalRequestId: string;
  originalEnvelopeId: string;
  replayedFinalState: 'ALLOW' | 'HOLD' | 'REJECT';
  replayedPermittedActionClass: string | null;
  matchedOriginalOutcome: boolean; // true iff states match
  replayedAt: string;
}
```

DiagnosticReportService (src/services/diagnostics/)

Generates a structured operator-facing diagnostic report from an EvidenceBundle. Includes: system state snapshot, invariant check summary, recommended actions (e.g. 'obtain human approval', 'check control status'), and a risk assessment. Used by the browser-demo support-readiness view.

```
interface DiagnosticReport {
  reportId: string;
  evidenceBundleId: string;
  systemState: { ... };
  invariantSummary: InvariantCheckSummary[];
  recommendations: string[];
  riskAssessment: string;
  generatedAt: string;
}
```

OperatorActionService (src/services/support/)

Provides operator-level actions: acknowledge a blocked decision, escalate a held request, request re-evaluation, generate a support ticket. All actions are recorded to the audit log.

SystemHealthService (src/services/support/)

Checks system preconditions: logging ready, control status valid, trust state valid, audit chain integrity. Returns a HealthReport with individual check results and an overall status.

SystemIntegrityService (src/services/support/)

Runs integrity checks across the audit log chain. Verifies that all entries are chain-linked, no gaps exist, and hash values are consistent. Used as a background health check and on export to confirm evidence is untampered before sharing.

CertificateFormatter (src/domain/formatters/)

Generates a human-readable governance certificate from an EvidenceBundle. Used by the auditor-view example to render a printable compliance record showing the decision, approver identity, invariants checked, and audit chain summary.

SECTION 9 — TEST SUITE — 16 FILES, 391 TESTS

All tests run with Vitest 2.1. Full suite completes in ~7.7 seconds. Zero mocks of the enforcement core — all tests use real ExecutionGateService instances.

Test File	Tests	Coverage focus
test/execution-gate-service.test.ts	~120	All 12 invariants; every ALLOW/HOLD/REJECT path; approval binding; timestamp validation: authority class validation
test/adversarial-integrity.test.ts	~18	Advanced attack scenarios: approval replay, cross-request binding, AI authority escalation, malformed envelope injection
test/audit-evidence-export.test.ts	~30	AppendOnlyLogService chain construction; EvidenceBundleService assembly; ExportManifestService output
test/persistent-audit-log.test.ts	~20	FileBackedAppendOnlyLogService: write, read-back, chain verify, multi-request interleave
test/diagnostic-report-service.test.ts	~18	DiagnosticReportService output structure; recommendation generation; risk assessment
test/snapshots/enforcement- loop.snapshot.test.ts	~8	Snapshot tests: full ALLOW/HOLD/REJECT flows serialized and compared across runs
test/security/fail-closed.test.ts	2	Unexpected non-CerbaSealError inputs produce REJECT — never ALLOW
test/security/non-forgery.test.ts	2	Self-constructed GateResult rejected by assertIsGateIssued (INV-06)
test/security/misuse-scenarios.test.ts	~15	AI self-authorization; authority-bearing proposals; approval authority class forgery
test/security/contextual-boundary.test.ts	~12	Boundary cases: empty strings, null fields, zero-length arrays, whitespace-only values
test/integration/full-flow.test.ts	1	End-to-end: request !' evaluate !' evidence bundle !' replay !' verify chain
test/integration/system-integration.test.ts	1	Full stack: gate + audit + evidence + diagnostic + health check + export
test/integration/browser-demo-routes.test.ts	~40	All 9 demo HTTP server routes respond 200 with correct content
test/integration/review-portal-routes.test.ts	~40	Review portal routes: /review, /pilot, /security, /deployment, /one-page
test/integration/support-readiness.test.ts	~30	Support readiness demo: all operator action scenarios
test/integration/external-signal- examples.test.ts	~34	External signal injection: trust state, control status, prohibited use, provenance

Test Commands

```
pnpm test           # run all 391 tests once
pnpm test:watch     # watch mode
pnpm check:invariants # verify 12/12 invariants have covering tests
pnpm audit:repo     # run all 15 repo audit checks (includes test count)
```


SECTION 10 — REPO AUDIT — 15 CHECKS EXPLAINED

pnpm audit:repo runs scripts/repo-audit.ts — a suite of 15 automated checks that verify the repo is in a releasable, self-consistent state. All 15 currently pass.

#	Check	What it verifies
1	Full test suite passes	Runs pnpm test — expects 391 tests, zero failures
2	TypeScript compiles without errors	Runs tsc --noEmit — zero type errors
3	README anchor strings present	Checks for 4 required anchor strings in README.md
4	All portal routes respond 200	Starts browser-demo server, requests 9 routes, expects 200
5	No src/ file unreferenced in tests or examples	Every file under src/ must appear in at least one test or example
6	Invariant registry exists and is non-empty	Loads invariants.ts — verifies 12 invariants defined
7	Known-limitations section present in README	README must contain '## Known Limitations' heading
8	Test count in README matches actual	README's 'N tests' claim must match actual test run count
9	demo:web:validate passes	Runs validate-demo.ts — all browser demo assertions pass
10	demo:support:validate passes	Runs validate-support-readiness.ts — 13 assertions pass
11	review:validate passes	Runs validate-review-portal.ts — 110 assertions pass
12	No architectural import boundary violations	Runs check-imports.ts — 0 violations across 79 files
13	All invariants linked to covering tests	Runs check-invariant-coverage.ts — 12/12 invariants covered
14	No stale test-count references in documentation	All doc references to test counts match actual
15	Pilot documents exist and contain required sections	3 pilot docs verified for required headings

Import Boundary Rules (check-imports.ts)

The following boundaries are enforced across all 79 files:

- examples/ and scripts/ must not import or from test/
- src

c/
m
us
t
n
ot
i
m
p
or
t f
ro
m
ex
a
m
pl
es
/
sc
ri
pt
s/
o
r t
es
t/
• t
es
t/
m
us
t
n
ot
i
m
p
or
t f
ro
m
ex
a
m
pl
es
/
or
s
cri
pt
s/
(e
xc
e
pt
fo
r s
h
ar
e

d
fix
tu
re
s)
•
N
o
cir
cu
lar
d
e
p
e
n
d
e
nc
ie
s
wi
thi
n
sr
c/
se
rvi
ce
s/

Proof Snapshot (export-proof.ts)

pnpm export:proof generates docs/reports/proof-snapshot.json — a signed manifest of the repo state including: test suite results, audit check results, invariant count, git commit hash, and a SHA-256 manifestChecksum. The manifest can be independently verified with pnpm verify:proof.

```
pnpm export:proof      # generate docs/reports/proof-snapshot.json
pnpm verify:proof      # verify the snapshot's manifestChecksum
pnpm generate:evidence-report # generate evidence package from snapshot
```


SECTION 11 — CLI SCRIPTS & COMMANDS

Script / Command	File	Purpose
pnpm test	vitest	Run all 391 tests
pnpm test:watch	vitest	Watch mode — re-runs on file changes
pnpm typecheck	tsc --noEmit	TypeScript type check without emitting
pnpm demo	examples/run-demo.ts	Console demo of all gate scenarios
pnpm demo:web	examples/browser-demo/server.ts	Start browser-based review portal (HTTP server)
pnpm demo:web:validate	examples/browser-demo/validate-demo.ts	Assert all demo routes and scenarios
pnpm demo:consumer	examples/consumer-example/consumer.ts	Consumer pattern demo
pnpm demo:agent	examples/agent-gate/run-agent-gate.ts	AI agent gate demo
pnpm demo:audit	examples/auditor-view/run-auditor-view.ts	Auditor certificate view demo
pnpm demo:support	examples/support-readiness/run-support-readiness.ts	Support readiness demo
pnpm demo:all	all above	Run all demos sequentially
pnpm check:imports	scripts/check-imports.ts	Verify import boundary rules (79 files)
pnpm check:invariants	scripts/check-invariant-coverage.ts	Verify 12/12 invariants have test coverage
pnpm export:proof	scripts/export-proof.ts	Generate proof-snapshot.json with SHA-256 checksum
pnpm verify:proof	scripts/verify-proof.ts	Verify proof-snapshot.json manifestChecksum
pnpm audit:repo	scripts/repo-audit.ts	Run all 15 repo audit checks
pnpm generate:pilot-config	scripts/generate-pilot-config.ts	Generate pilot config package from wizard-input.json
pnpm generate:evidence-report	scripts/generate-evidence-report.ts	Generate evidence package from proof-snapshot.json
pnpm review:validate	examples/browser-demo/validate-review-portal.ts	Assert 110 review portal scenarios

generate:pilot-config — Output Files

Reads wizard-input.json from project root.
Writes to pilot-config/ directory:

cerbaseal-config.json: Ready-to-use runtime config for the client's workflow and authority classes
pilot-checklist.md: Phase-by-phase onboarding checklist with owner and acceptance criteria per item
scenario-test.ts: Runnable TypeScript test of the client's specific workflow (approve/reject scenarios)
deployment-summary.md: One-page deployment summary with environment variables, pre-flight checklist, and next steps

generate:evidence-report — Output Files

Reads docs/reports/proof-snapshot.json. Writes to evidence-package/ directory:
governance-summary.md: Narrative compliance evidence: test results, audit checks, invariant coverage, chain validity
decision-summary.json: Machine-readable enforcement state: gate config, invariants, proof snapshot metadata
audit-integrity-summary.md: Hash chain verification details: manifest checksum status, HMAC signature status

SECTION 12 — INTEGRATION STARTER KITS (×4)

Four complete working examples in `examples/`. Each runs standalone with `pnpm tsx`. Each has a `README.md` explaining the pattern and how to adapt it.

rest-api-starter · **examples/rest-api-starter/**

Pattern: HTTP REST wrapper — expose the gate as an API

A Node.js HTTP server (no framework) that wraps `ExecutionGateService` as REST endpoints. Useful as a sidecar or internal service that other systems call over HTTP.

•
P
O
S
T
/e
va
lu
at
e
—
e
va
lu
at
e
a
G
ov
er
n
e
d
R
e
q
u
es
t,
re
tu
rn
s
G
at
e
R
es
ult

+
E
vi
d
e
nc
e
B
u
n
dl
e
•

G
E
T
/
d
ec
isi
o
ns

—
li
st
all
d
ec
isi
o
ns
fr
o
m
th
e
in-
m
e
m
or
y
a
u
dit
lo
g
•

G
E
T
/
d
ec
isi
o
ns
/:i
d
—
g
et

a
sp
ec
ifi
c
d
ec
isi
o
n
by

r
e
q
u
es
tl
d
•
G
E
T
/
h
e
alt
h
—
s
ys
te
m
h
e
alt
h
ch
ec
k(
lo
g
gi
n
g
re
a
dy
, c
h
ai
n
va
lid
, c
o
nt
rol
st
at
us
)

```
pnpm tsx examples/rest-api-starter/index.ts  
curl -X POST http://localhost:3001/evaluate -d @examples/rest-api-starter/sample-request.json
```

financial-approval-starter · examples/financial-approval-starter/

Pattern: AI proposes !' analyst holds !' manager approves !' ALLOW

Demonstrates the two-step human approval pattern for financial escalations. An AI flags a high-risk transaction; an analyst reviews; a manager provides the final ApprovalArtifact. Shows the full HOLD !' ALLOW lifecycle with evidence bundle generation.

•
St

e
p
1:
S
u
b
m
i
t
w
i
t
h
o
u
t
a
p
r
o
v
a
l
! ' H
O
L
D
(R
E
Q
U
I
R
E
D
-
A
P
P
R
O
V
A
L
-
M
I
S
S
I
N
G)
•
St
e
p
2:
M
a
n
a
g
e
r
e
v
i
e
w
s
a
n
d

co
ns
tr
uc
ts
A
p
pr
ov
al
Ar
tif
ac
t

•
St
e
p
3:
R
es
u
b
mi
t
wi
th
a
rti
fa
ct
!' A
L
L
O
W

+
R
el
e
as
e
A
ut
h
or
iz
ati
o
n

•
St
e
p
4:
G
e
n
er
at
e

E
vi
d
e
nc
e
B
u
n
dl
e
wi
th

3-
ev
e
nt
c
h
ai
n

•

St
e
p
5:
V
er
ify

:

Al
al
o
n
e

!' R

E

J

E

C

T

(A

I_

C

A

N

N

O

T

—

A

U

T

H

O

RI

Z

E)

pnpm tsx examples/financial-approval-starter/index.ts

fraud-workflow-starter · examples/fraud-workflow-starter/

Pattern: AI-scored triage with persistent file-backed audit log

Processes transactions through risk scoring (0–100) with risk-level routing. Uses FileBackedAppendOnlyLogService for durable JSONL audit logs. Demonstrates fraud_triage unconditional approval enforcement.

•
L
o
w
ris
k (
<
5
0)
: a
llo
w
ac
tio
n,
n
o
a
p
pr
ov
al
—
b
ut
fr
a
u
d
_t
ri
a
g
e
ov
er
ri
d
es
(
H
O
L
D)
•
M
e
di
u
m
ris
k (
5
0
—
7
9)
: h

ol
d
ac
tio
n,
a
n
al
ys
t
a
p
pr
ov
al
re
q
uir
e
d
!' A
L
L
O
W
•
Hi
g
h
ris
k
("e
8
0)
: e
sc
al
at
e
ac
tio
n,
c
o
m
pli
a
nc
e
—
of
fic
er
a
p
pr
ov
al
re
q
uir
e

d
!' A
L
L
O
W
•
A
u
dit
lo
g:
./
a
u
dit
/fr
a
u
d-
tri
a
g
e.j
so
nl
(J
S
O
N
L,
p
er
si
st
s
ac
ro
ss
r
u
ns
)

```
pnpm tsx examples/fraud-workflow-starter/index.ts
```

agent-integration-starter · examples/agent-integration-starter/

Pattern: AI agent proposes !' system actor carries !' human approves

Shows the correct pattern for integrating an AI agent: the AI never submits its own proposal as actor (INV-05 REJECT).

Instead, a system actor carries the AI proposal. A human reviewer then provides the ApprovalArtifact. Includes a deliberate wrong-pattern demo showing why AI self-authorization always produces REJECT.

•
S
ce
n
ar
io
1:
AI
a
ct

or
s
u
b
mi
ts
o
w
n
pr
o
p
os
al
!' R
E
J
E
C
T
(A
L
C
A
N
N
O
T
—
A
U
T
H
O
R
I
Z
E)
•
S
ce
n
ar
io
2:
S
ys
te
m
ac
to
r c
ar
ri
es
A
l
pr
o
p
os
al
!' H

O
L
D
(w
ait
in
g
fo
r
h
u
m
a
n)

•
S
ce
n
ar
io
3:
H
u
m
a
n
re
vi
e
w
er
a
p
pr
ov
es
!'
A
L
L
O
W

+
E
vi
d
e
nc
e
B
u
n
dl
e
•
E
vi
d
e
nc
e:

3-
ev
e
nt
c
h
ai
n
wi
th
m
o
d
el
ve
rsi
o
n
+
re
vi
e
w
er
id
e
nti
ty

```
pnpm tsx examples/agent-integration-starter/index.ts
```

SECTION 13 — ADOPTION LAYER — 5 PRIORITIES

Five adoption-layer deliverables built in response to the Line Axia audit. Together they allow a client to onboard, configure, pilot, and generate compliance evidence without requiring Jesse to be present for every step.

PRIORITY 1

Authority Class Registry

New client authority classes (e.g. 'risk_officer') added to cerbaseal.config.json with zero TypeScript source changes. ExecutionGateService constructor accepts optional GateConfig. Default (no-args) constructor uses core classes only — zero breaking changes.

Files: cerbaseal.config.json · src/config/cerbaseal-config.ts

! Eliminates: 'Jesse must update the code every time we add a role'

PRIORITY 2

Integration Starter Kits

Four complete working examples covering the most common integration patterns. Each runs standalone, includes a README, and demonstrates the correct actor/approver pattern. Client developers can copy and adapt without needing to understand the full codebase.

Files: examples/rest-api-starter/ · examples/financial-approval-starter/ · examples/fraud-workflow-starter/ · examples/agent-integration-starter/

! Eliminates: 'We need Jesse on a call to show us how to integrate'

PRIORITY 3

Workflow Config Generator

Client fills in wizard-input.json (wizard-style questionnaire about their workflow, roles, approval rules, environment). pnpm generate:pilot-config produces a complete 4-file pilot configuration package: cerbaseal-config.json, pilot-checklist.md, scenario-test.ts, deployment-summary.md.

Files: scripts/generate-pilot-config.ts · scripts/wizard-input.example.json

! Eliminates: 'Jesse needs to configure this for each client'

PRIORITY 4

Pilot Evidence Package Generator

After a successful pilot, run pnpm export:proof then pnpm generate:evidence-report to produce a compliance evidence package (3 files) from the cryptographically verified proof snapshot. Suitable for sharing with compliance officers, regulators, or auditors.

Files: scripts/generate-evidence-report.ts

! Eliminates: 'Jesse must produce the compliance evidence manually'

PRIORITY 5

Founder Independence Kit

Single-page master index covering all 7 adoption stages from client qualification to compliance review. 8-phase onboarding sequence a client follows end-to-end without Jesse. Includes escalation table, common errors guide, and explicit list of what still requires Jesse.

Files: docs/FOUNDER-INDEPENDENCE-KIT.md · docs/client-adoption/onboarding-sequence.md

! Eliminates: 'The whole sales and onboarding process depends on Jesse's time'

SECTION 14 — DEMO APP ROUTES

The browser-demo server (examples/browser-demo/server.ts) is a plain Node.js HTTP server with no web framework. It serves both HTML portal pages and JSON API endpoints. Start with: `pnpm demo:web` (default port: 3000 or \$PORT)

Portal Pages (HTML — for human review)

Route	Title	Content
<code>/review</code>	CerbaSeal Review Portal	Interactive governance decision review with live scenario evaluation
<code>/pilot</code>	Pilot Readiness	Pilot readiness checklist and deployment status dashboard
<code>/security</code>	Security Summary	Invariant summary, adversarial test results, security posture overview
<code>/deployment</code>	Deployment Guide	Step-by-step deployment instructions and environment configuration
<code>/one-page</code>	One-Page Summary	Executive-level one-page overview of CerbaSeal governance model

Scenario APIs (JSON — return gate decisions)

Route	Scenario	Expected outcome
<code>POST /api/reject</code>	AI actor attempts self-authorization	REJECT — AI_CANNOT_AUTHORIZE
<code>POST /api/hold</code>	System actor, approval required but missing	HOLD — REQUIRED_APPROVAL_MISSING
<code>POST /api/allow</code>	System actor, approval present and valid	ALLOW — DECISION_ALLOWED

Data APIs (JSON — return structured data for portal)

Route	Returns
<code>/api/review-summary</code>	Recent decisions, chain validity, invariant check summary
<code>/api/pilot-readiness</code>	Pilot checklist items with status, deployment mode, configuration summary
<code>/api/security-summary</code>	Invariant list with test coverage, adversarial test results, security score

Validation Scripts

```
pnpm demo:web:validate # validate-demo.ts – exercises all routes, checks JSON structure
pnpm review:validate   # validate-review-portal.ts – 110 assertions across all portal routes
```

cerbaseal-demo Artifact (Vite + React)

A separate React/Vite application in artifacts/cerbaseal-demo/ provides a production-grade browser UI. Registered as a pnpm workspace artifact with its own dev server. Start with: `pnpm --filter @workspace/cerbaseal-demo run dev`

SECTION 15 — FULL REPOSITORY FILE TREE

All non-node_modules files organized by directory.

package.json Root package — 18 scripts,
tsx + pdfkit + vitest devDependencies
tsconfig.json Strict TypeScript — ES2022
target, NodeNext module, strict: true
cerbaseal.config.json Runtime authority/
workflow/action class registry
vitest.config.ts Vitest configuration —
include: test/**/*test.ts
.gitignore Excludes: wizard-input.json,
pilot-config/, evidence-package/, audit/
README.md Full public README with
adoption layer section
CERBASEAL_PILOT_READINESS_BINDER.md
Pilot readiness binder for client delivery

src/

config/

cerbaseal-config.ts
loadCerbaSealConfig(),
buildAllowedAuthorityClasses(), GateConfig
interface

domain/

types/core.ts All primary types:
GovernedRequest, DecisionEnvelope,
ApprovalArtifact, GateResult, ...
types/audit.ts Audit types: AuditLogEntry,
EvidenceBundle, ExportManifest, ReplayResult
types/diagnostics.ts DiagnosticReport,
HealthReport, SystemState types
constants/invariants.ts INVARIANTS
object — INV-01 through INV-12
constants/reason-codes.ts
REASON_CODES object — 18 reason codes
errors/cerbaseal-error.ts
CerbaSealError class — carries invariant,
reasonCode, finalState
formatters/certificate.ts
CertificateFormatter — human-readable
governance certificate
formatters/demo-response.ts
DemoResponseFormatter — browser demo
JSON shape
review-portal-data.ts Data builders for
review portal API endpoints
builders/agent-scenarios.ts Agent
gate test scenario builders
builders/consumer-scenarios.ts
Consumer example scenario builders
builders/gate-scenarios.ts Core gate
scenario builders (ALLOW/HOLD/REJECT)
builders/request-fixtures.ts
GovernedRequest fixture builders for tests

services/execution/

execution-gate-service.ts
ExecutionGateService — the mandatory
enforcement gate
mock-execution-system.ts
MockExecutionSystem — test helper for
simulating callers
tools.ts Shared gate utility functions

services/audit/

append-only-log-service.ts
AppendOnlyLogService (in-memory) +
IAuditLogService interface
**file-backed-append-only-log-
service.ts**
FileBackedAppendOnlyLogService (JSONL
persistence)
audit-hash-utils.ts SHA-256 chain
construction and verification

services/evidence/

evidence-bundle-service.ts
EvidenceBundleService — assembles full

- decision context
- services/export/**
 - export-manifest-service.ts**
ExportManifestService — point-in-time evidence hash snapshot
- services/replay/**
 - replay-service.ts** ReplayService — determinism verification for past decisions
- services/diagnostics/**
 - diagnostic-report-service.ts**
DiagnosticReportService — operator analysis + recommendations
- services/support/**
 - operator-action-service.ts** Operator-level actions: acknowledge, escalate, re-evaluate
 - system-health-service.ts**
SystemHealthService — precondition health checks
 - system-integrity-service.ts**
SystemIntegrityService — audit chain integrity verification
 - support-fixtures.ts** Test fixtures for support services
- test/**
- execution-gate-service.test.ts** ~120 tests — all invariants and gate paths
- adversarial-integrity.test.ts** ~18 tests — advanced attack scenarios
- audit-evidence-export.test.ts** ~30 tests — audit chain and evidence assembly
- persistent-audit-log.test.ts** ~20 tests — file-backed log
- diagnostic-report-service.test.ts** ~18 tests — diagnostic report
- security/fail-closed.test.ts** 2 tests — unexpected exceptions !' REJECT
- security/non-forgery.test.ts** 2 tests — WeakSet bypass prevention
- security/misuse-scenarios.test.ts** ~15 tests — AI self-auth, forgery attempts
- security/contextual-boundary.test.ts** ~12 tests — edge cases and boundary values
- snapshots/enforcement-loop.snapshot.test.ts** ~8 tests — snapshot comparisons
- integration/full-flow.test.ts** 1 test — full end-to-end flow
- integration/system-integration.test.ts** 1 test — full stack integration
- integration/browser-demo-routes.test.ts** ~40 tests — HTTP route assertions
- integration/review-portal-routes.test.ts** ~40 tests — portal route assertions
- integration/support-readiness.test.ts** ~30 tests — support readiness scenarios
- integration/external-signal-examples.test.ts** ~34 tests — external signal injection
- examples/**
- run-demo.ts** Console demo — all gate scenarios
- http-wrapper.ts** HTTP wrapper utility
- rest-api-starter/** REST API starter kit (index.ts + README.md + sample-request.json)
- financial-approval-starter/** Financial approval starter kit (index.ts + README.md)
- fraud-workflow-starter/** Fraud workflow starter kit (index.ts + README.md)
- agent-integration-starter/** Agent integration starter kit (index.ts + README.md)
- browser-demo/** HTTP review portal server (server.ts, scenarios.ts, review-portal.ts, ...)

consumer-example/ Consumer pattern example
agent-gate/ AI agent gate example
auditor-view/ Auditor certificate view
support-readiness/ Support readiness example

scripts/

repo-audit.ts 15-check automated repo audit
export-proof.ts Generate proof-snapshot.json with SHA-256 manifest checksum
verify-proof.ts Verify proof-snapshot.json
check-imports.ts Import boundary checker (79 files)
check-invariant-coverage.ts Invariant-to-test coverage checker
generate-pilot-config.ts Pilot configuration package generator
generate-evidence-report.ts Evidence package generator
wizard-input.example.json Example wizard-input.json for fraud triage workflow
hash-report.ts Report hash utility

docs/

FOUNDER-INDEPENDENCE-KIT.md Master index for all 7 adoption stages
client-adoption/onboarding-sequence.md 8-phase client onboarding sequence
client-adoption/ 20+ client-facing guides, playbooks, templates, training materials
deployment/ Deployment modes, EU pilot posture, runbook
security/ Access control, artifact signing roadmap, security review brief
reports/ Adversarial integrity reports, CTO review pack, proof-snapshot.json

artifacts/cerbaseal-demo/

src/ React + Vite browser UI (hooks, utils, components)
vite.config.ts Vite configuration — reads PORT env var
package.json @workspace/cerbaseal-demo

specs/

approval_artifact.json JSON schema for ApprovalArtifact
approval_artifact.md Specification document for ApprovalArtifact
